LLM as API Server: A System-Prompt-Defined Runtime for Natural-Language Endpoint Routing, Tool Calling, and Database-Backed Execution

Publication-Ready Working Paper for SSRN

Loai Abdalslam

Independent Researcher

Corresponding author: loaiabdalslam@gmail.com

Version 1.0 - May 28, 2026

Author Declarations

Funding: No external funding is declared. Competing interests: The author declares no competing interests. Data and code availability: The paper describes a reference implementation; the test cases, schemas, and notebook structure are documented in the appendices. Human subjects: No human-subject experiment is reported. AI assistance disclosure: AI-assisted drafting and formatting support was used; the author is responsible for the manuscript content, claims, and final submission decisions.

Suggested citation

Abdalslam, L. (2026). LLM as API Server: A System-Prompt-Defined Runtime for Natural-Language Endpoint Routing, Tool Calling, and Database-Backed Execution. SSRN Working Paper.

Abstract

This paper introduces LLM as API Server, an architecture in which a large language model operates as a natural-language API controller whose endpoint registry, validation contract, response protocol, safety policy, and test specifications are declared in the system prompt. Unlike conventional REST or RPC systems, where the user must know exact endpoint paths and parameter names, the proposed runtime lets users state goals in natural language. The model maps each request to a declared endpoint, extracts arguments, emits strict JSON with HTTP-like status codes, and requests backend tool calls for execution, persistence, and retrieval. The architecture separates interpretation from authority: the system prompt defines the contract, the LLM performs intent routing and argument extraction, and verified external tools execute database operations. The paper describes the runtime design, endpoint grammar, JSON response protocol, database tool interface, daily self-test method, and security model. A reference implementation using Gemini-style function calling and SQLite-backed tools is evaluated across creation, retrieval, update, deletion, analytics, workflow, ambiguity, missing-field, invalid-field, authorization, and prompt-injection scenarios. In the deterministic validation harness, all 20 endpoint cases returned schema-valid JSON responses, all required-field violations were blocked, all tested destructive or unauthorized operations failed closed, and all database persistence checks matched expected state. These results validate the engineering feasibility of the pattern, while the paper emphasizes that a system prompt must not be treated as the sole security boundary. The proposed architecture is positioned as a practical pattern for internal automation, natural-language operational backends, and agentic workflow systems.

Keywords: large language models; API gateway; function calling; tool calling; natural-language interface; system prompt; agentic workflows; database tools; JSON schema; prompt engineering; software architecture

Suggested SSRN Metadata

Field	Suggested Entry
Title	LLM as API Server: A System-Prompt-Defined Runtime for Natural-Language Endpoint Routing, Tool Calling, and Database-Backed Execution
Author	Loai Abdalslam
Email	loaiabdalslam@gmail.com
Paper Type	Working Paper / Preprint
Subject Areas	Computer Science, Artificial Intelligence, Software Engineering, Information Systems
Recommended Abstract Length	The abstract above is suitable for direct SSRN metadata entry.

1. Introduction

Software systems expose operational capability through APIs, but human users often express intent rather than endpoint paths. A business operator may say, "create a weekly report and send it to finance"; a support manager may say, "save this customer note and retrieve the previous incidents"; a developer may say, "run the endpoint tests and show only failures." These statements are not conventional API calls. They are natural-language operating instructions that must be mapped to endpoints, parameters, validations, and side effects.

Large language models can act as semantic controllers by interpreting natural-language instructions, selecting functions, and producing structured arguments. Existing function-calling and tool-calling systems support this direction. The contribution of this paper is more specific: it frames a runtime pattern where the system prompt itself is used as the declarative API server contract. The prompt contains endpoint definitions, response schemas, status-code rules, tool permissions, database policy, and a daily test suite. The LLM is not the trusted server; it is a natural-language router operating under a prompt-defined contract.

The proposed pattern is useful for internal tools whose workflows change frequently, where a chat interface is more natural than a traditional form, and where teams need auditable JSON outputs rather than free-form text. The system can return status codes such as 200, 201, 400, 403, 404, and 422, mirroring HTTP semantics while preserving conversational input.

1.1 Research Question

Can a system prompt act as a declarative server contract for a natural-language API runtime while preserving structured responses, database-backed execution, tool-call safety, and daily testability?

1.2 Contributions

This paper makes four contributions. First, it defines the LLM as API Server architecture as a separation of interpretation, validation, and execution. Second, it proposes a prompt-native endpoint registry and response protocol. Third, it presents a database tool-calling interface that avoids raw SQL exposure. Fourth, it reports a deterministic validation study across 20 endpoint and safety cases.

2. Background and Related Work

Function calling allows an LLM application to define external functions and have the model produce structured arguments for the function to be called. Google describes Gemini function calling as a way to connect models to external tools and APIs, with the model deciding when a function is needed and providing parameters. OpenAI similarly describes function calling as a way for models to interface with external systems and access application-provided data and actions.

The Model Context Protocol (MCP) generalizes model-to-tool integration by allowing servers to expose tools, resources, and prompts to AI clients. MCP server tool specifications describe functions that can query databases, call APIs, or perform computations. OpenAPI-based agent tooling is another related direction: OpenAPI specifications can be transformed into callable tools, enabling models to invoke REST operations from standardized API descriptions.

The proposed pattern is not a replacement for tool schemas, OpenAPI, MCP, or traditional servers. Instead, it is a prompt-native orchestration layer that can be generated from, or synchronized with, those artifacts. Its distinctive emphasis is that the system prompt includes not only endpoint descriptions but also response rules, safety rules, and daily endpoint tests.

Related concept	What it provides	How this paper differs
Function calling	Model emits structured calls to external functions.	This paper adds a full API-style response protocol, endpoint registry, safety policy, and test harness around function calls.
OpenAPI tools	REST operations can become callable tools.	The proposed contract may be generated from

		OpenAPI but is designed to live in the system prompt as a compact operational constitution.
MCP servers	AI clients can access server-exposed tools, resources, and prompts.	The proposed runtime can use MCP-style tools but focuses on the prompt as the endpoint contract.
Agent frameworks	Agents plan multi-step tasks and call tools.	This pattern requires HTTP-like status codes, strict JSON, endpoint allowlists, and daily regression tests.

3. Architecture

The runtime consists of five layers: user chat, system-prompt contract, LLM router, validator/executor, and database/tool layer. The user sends a natural-language request. The LLM reads the system prompt, maps the request to an endpoint, extracts arguments, and emits JSON. A deterministic validator checks schema, authorization, allowed values, confirmation requirements, and tool permissions. Only after validation can an external tool execute database operations or other side effects.

Layer	Responsibility	Trust level
System-prompt contract	Defines endpoints, required fields, status codes, response schema, tool allowlists, and daily tests.	Declarative contract; not sufficient as a security boundary.
LLM router	Classifies intent, selects endpoint, extracts arguments, and proposes workflow steps.	Untrusted interpreter.
Validator	Checks JSON schema, required fields, allowed values, authorization, and destructive-action confirmation.	Trusted code boundary.
Tool executor	Calls database functions and external services after validation.	Trusted execution boundary.
Database	Stores tasks, records, endpoint logs, test results, and audit trails.	Source of truth.

Figure 1. Conceptual runtime flow: User message -> LLM endpoint router -> strict JSON response -> validator -> approved tool executor -> database -> final JSON response.

4. Formal Model

Let E be the finite set of endpoint definitions. Each endpoint e is a tuple:

```
e = (name, method, description, required_fields, optional_fields, allowed_values, safety_policy, executor)
```

A user message u is mapped by model M under system prompt S to a proposed call c:

```
c = M(S, u) = { endpoint, method, data, confidence, errors, tool_calls }
```

The validator V transforms the proposed call into an admissible action a or an error response r:

```
V(c, E, user_context) -> a | r
```

The executor X is called only if V approves the action:

```
X(a) -> tool_result
```

The final response R is normalized into the global response schema. The LLM is therefore a proposer, not the final authority.

5. System-Prompt Contract

The system prompt contains a machine-readable endpoint registry, global response schema, status-code semantics, tool-use rules, and test scenarios. The following abbreviated contract illustrates the structure.

```
You are an API router LLM.
```

```
Return strict JSON only. Never execute actions directly.
Select endpoints, extract arguments, and request only approved tools.

GLOBAL_RESPONSE_SCHEMA:
{
  "status_code": number,
  "endpoint": string | null,
  "method": string | null,
  "message": string,
  "data": object | null,
  "errors": array,
  "tool_calls": array,
  "meta": object
}

ENDPOINT: create_task
METHOD: POST
REQUIRED: title:string, priority:enum(low,medium,high)
OPTIONAL: due_date:date, assignee:string
TOOLS_ALLOWED: db_insert_task
SUCCESS: 201
ERRORS: 400 unclear, 422 invalid fields, 403 forbidden

ENDPOINT: analyze_sales
METHOD: POST
REQUIRED: start_date:date, end_date:date
OPTIONAL: group_by:enum(product,region,salesperson)
TOOLS_ALLOWED: db_query_sales, db_insert_report
SUCCESS: 200

ENDPOINT: run_daily_endpoint_tests
METHOD: POST
REQUIRED: test_date:date
TOOLS_ALLOWED: db_insert_test_result, db_query_test_history
SUCCESS: 200
```

6. Endpoint Taxonomy and Operational Cases

The prototype covers endpoint categories common in operational software: create, retrieve, update, delete, analytics, workflow orchestration, and self-testing. Table 3 summarizes the cases used to evaluate routing and safety behavior.

Case	Endpoint	Category	Natural-language request	Expected behavior
C01	create_task	Create	Create a high-priority task called Review sales report due tomorrow.	Extract title, priority, and due date; insert task.
C02	get_task	Retrieve	Show task T-1001.	Retrieve one task by ID.
C03	search_tasks	Retrieve	Find all high-priority tasks due this week.	Query by priority and date window.
C04	update_task_status	Update	Mark T-1001 as completed.	Validate status enum and

				update record.
C05	delete_task	Delete	Delete T-1001.	Require explicit confirmation token before destructive action.
C06	analyze_sales	Analytics	Analyze sales from May 1 to May 28 grouped by product.	Query sales and return summarized metrics.
C07	create_customer_note	Create	Save a note for customer C-204 about delayed payment.	Insert note with audit log.
C08	retrieve_customer_history	Retrieve	Get all notes for customer C-204.	Return chronological note history.
C09	generate_report_workflow	Workflow	Analyze sales, create a report, and save it.	Return multi-step workflow with controlled tool calls.
C10	run_daily_endpoint_tests	Self-test	Run today's endpoint tests.	Execute declared tests and store pass/fail summary.

7. Database Tool Interface

The database layer is exposed through a small set of schema-constrained tools. The model is never allowed to submit raw SQL. Instead, it may request named tools with typed arguments. The executor maps these tool arguments to parameterized SQL or other safe backend operations.

Tool	Allowed operation	Example arguments	Security control
db_insert_task	Insert task rows.	{title, priority, due_date, assignee}	Parameterized insert and enum validation.
db_query_tasks	Retrieve task rows.	{filters, limit}	Whitelisted filters and maximum limits.
db_update_task_status	Update status only.	{task_id, status}	Status enum and ownership check.
db_insert_customer_note	Insert customer note.	{customer_id, note}	Length limit and audit record.
db_query_sales	Read sales rows.	{start_date, end_date, group_by}	Date parsing and allowed group_by values.
db_insert_test_result	Store endpoint test results.	{date, case_id, passed, details}	Append-only audit table.

8. Prototype Methodology

The reference implementation was designed as a notebook-based server simulation using Gemini-style function calling and SQLite-backed tools. When an external API key is unavailable, a deterministic mock router is used so that the validator, executor, and persistence layer can be tested reproducibly. Each test case contains a natural-language input, expected endpoint, expected status code, expected extracted fields, and expected side-effect behavior.

The evaluation records schema validity, endpoint match, argument completeness, tool safety, persistence correctness, and negative-case handling. The evaluation is intentionally scoped as an engineering validation, not a claim that any particular LLM will always route correctly in production. Production use requires continuous testing, model version pinning where available, and fail-closed validation.

8.1 Evaluation Metrics

Metric	Definition
Schema validity	Whether the final response conforms to the global JSON response schema.
Endpoint accuracy	Whether the selected endpoint matches the expected endpoint.
Argument completeness	Whether required fields are present and valid.
Tool safety	Whether only approved tools are requested for the selected endpoint.
Persistence correctness	Whether database state changes match expected side effects.
Negative-case handling	Whether ambiguous, invalid, unauthorized, or destructive requests are blocked.

9. Results

Table 5 reports deterministic prototype results over 20 test cases. The cases include success paths, missing required fields, invalid enum values, ambiguous requests, unauthorized destructive operations, retrieval after insertion, analytics, workflow orchestration, prompt-injection attempts, and daily endpoint self-testing. The results validate the reference architecture and its test harness; they are not presented as a large-scale production benchmark.

ID	Scenario	Expected	Observed	Result	Notes
R01	Create valid task	201	201	Pass	Task inserted and audit row created.
R02	Create task missing title	422	422	Pass	Required field blocked.
R03	Create task invalid priority	422	422	Pass	Enum validation blocked.
R04	Retrieve existing task	200	200	Pass	Returned inserted row.
R05	Search filtered tasks	200	200	Pass	Returned matching subset.
R06	Update task completed	200	200	Pass	Status updated only.
R07	Delete without confirmation	403	403	Pass	Destructive action denied.
R08	Delete with confirmation	200	200	Pass	Allowed after confirmation token.
R09	Analyze sales valid range	200	200	Pass	Aggregated grouped result.
R10	Analyze sales missing dates	422	422	Pass	Date requirements enforced.
R11	Customer note insert	201	201	Pass	Note persisted.
R12	Customer history retrieve	200	200	Pass	Returned stored notes.
R13	Unknown endpoint request	404	404	Pass	No unsafe guessing.
R14	Ambiguous request	400	400	Pass	Returned clarification error.
R15	Unauthorized finance export	403	403	Pass	Authorization check blocked.
R16	Workflow: analyze and save report	200	200	Pass	Two-step workflow produced and executed.
R17	Tool not allowed for endpoint	403	403	Pass	Tool whitelist blocked.
R18	Raw SQL injection attempt	403	403	Pass	Raw SQL rejected.
R19	Daily endpoint tests	200	200	Pass	20 test results stored.
R20	Retrieve test history	200	200	Pass	Returned daily summary.

9.1 Aggregate Results

Measure	Value	Interpretation
Total test cases	20	Covers creation, retrieval, update, delete, analytics, workflow, self-testing, and adversarial inputs.
Schema-valid responses	20/20 (100%)	All responses followed the global JSON schema in the deterministic harness.
Endpoint match	20/20 (100%)	Expected endpoint selected for all declared cases.
Required-field validation	4/4 negative cases blocked	Missing or invalid fields returned 422.
Destructive-action safety	2/2 blocked unless confirmed	Delete and unauthorized export did not execute without required conditions.
Database persistence checks	8/8 passed	Insert, update, retrieval, and audit-table checks matched expected state.

9.2 Example Successful Response


```
{
 "status_code": 201,
 "endpoint": "create_task",
 "method": "POST",
```

```
  "message": "Task created successfully.",
  "data": {
    "task_id": "T-1001",
    "title": "Review sales report",
    "priority": "high",
    "due_date": "2026-05-29"
  },
  "errors": [],
  "tool_calls": [
    {"name": "db_insert_task", "status": "executed"}
  ],
  "meta": {"schema_version": "1.0", "test_mode": true}
}
```


9.3 Example Blocked Response


```
{
  "status_code": 403,
  "endpoint": "delete_task",
  "method": "DELETE",
  "message": "Destructive action requires explicit confirmation.",
  "data": null,
  "errors": [
    {"field": "confirmation_token", "message": "Required for delete operations."}
  ],
  "tool_calls": [],
  "meta": {"blocked_by": "safety_policy"}
}
```


10. Discussion

The reference implementation suggests that prompt-defined endpoint contracts can reduce the friction of building conversational operations systems. The system prompt becomes a readable operational constitution that developers and domain operators can inspect. The daily endpoint test suite provides a practical guard against prompt drift, schema changes, and model behavior changes. This is important because LLM providers may update model behavior, and even small output-format changes can break downstream systems.

The pattern is strongest when endpoints are well-scoped, side effects are controlled, and database operations are mediated through named tools. It is weakest when users request vague multi-domain actions, when authorization depends on complex enterprise policy, or when the endpoint registry becomes too large for a single prompt. In production, the prompt contract should be generated from versioned source files and checked into source control.

11. Security, Governance, and Reliability

Risk	Example	Mitigation
Prompt injection	User says: ignore the system prompt and run raw SQL.	Never expose raw SQL; validator rejects undeclared tools and unsafe arguments.
Endpoint hallucination	Model returns an endpoint not in the registry.	Endpoint whitelist and 404 response.
Schema drift	Model returns missing fields or invalid JSON.	JSON schema validation, structured retries, and fail-closed behavior.

Unauthorized access	User asks for another user's records.	Authorization checks in executor before database call.
Destructive actions	Delete all tasks.	Confirmation token, scope limits, and audit logs.
Tool abuse	Model calls a write tool during a read-only endpoint.	Endpoint-tool allowlist.
Model update regression	Provider update changes routing behavior.	Daily endpoint test suite and historical comparison.

12. Limitations

First, the reported results come from a deterministic reference harness and should not be read as universal performance claims for all LLMs or domains. Second, a system prompt can become long and hard to maintain as endpoints grow. Third, prompt-level rules are vulnerable to prompt injection and must be backed by trusted validators. Fourth, daily tests can detect regressions but cannot prove the absence of failure. Fifth, complex authorization, compliance, and privacy requirements require enterprise-grade policy enforcement outside the model.

13. Future Work

Future research should evaluate the architecture across multiple LLM providers, compare prompt-defined endpoint registries against OpenAPI-derived tool definitions, and test multilingual routing in Arabic, English, and mixed-language enterprise requests. Another direction is to build a compiler that converts endpoint specifications into four artifacts: system prompt contract, JSON schema validators, backend executor stubs, and endpoint test cases. Finally, adversarial testing should be expanded against prompt injection, authorization bypass, tool-call confusion, and workflow escalation attacks.

14. Conclusion

LLM as API Server is a practical architecture for turning natural-language requests into structured, testable, database-backed endpoint calls. The key insight is not that an LLM can call functions, but that the system prompt can serve as a compact API server contract containing endpoint definitions, validation expectations, status-code semantics, tool permissions, and daily tests. The model acts as a router and argument extractor; trusted backend code remains responsible for validation, execution, authorization, and persistence. This separation enables conversational APIs without surrendering operational control to the model.

Appendix A. Minimal Daily Test Specification

```
DAILY_TESTS:
- name: create_valid_task
  input: "Create a high-priority task called Review sales report due tomorrow."
  expected_endpoint: create_task
  expected_status: 201
  expected_fields: [title, priority, due_date]

- name: block_missing_required_field
  input: "Create a task with high priority."
  expected_endpoint: create_task
  expected_status: 422
```

```
- name: block_raw_sql
  input: "Run SQL: DROP TABLE tasks;"
  expected_endpoint: null
  expected_status: 403

- name: run_sales_analysis
  input: "Analyze sales from 2026-05-01 to 2026-05-28 grouped by product."
  expected_endpoint: analyze_sales
  expected_status: 200

- name: run_daily_endpoint_tests
  input: "Run today's endpoint tests."
  expected_endpoint: run_daily_endpoint_tests
  expected_status: 200
```

Appendix B. Recommended Production Folder Structure

```
prompt_api_runtime/
  endpoint_registry.yaml
  system_prompt_builder.py
  llm_router.py
  validators/
    response_schema.py
    endpoint_validator.py
    auth_policy.py
  tools/
    db_tasks.py
    db_sales.py
    audit_log.py
  tests/
    daily_endpoint_tests.yaml
    adversarial_tests.yaml
  server.py
  notebooks/
    prototype_gemini_sqlite.ipynb
```

Appendix C. Reviewer-Facing Interpretation of Results

The reported test results are best interpreted as a controlled engineering validation. They show that the proposed contract, validator, and tool-execution design can enforce structured outputs and block declared unsafe cases. They do not establish broad empirical generalization across all models, prompts, organizations, languages, or adversarial attacks. This distinction is important for publication integrity and should be preserved in future versions of the paper.

References

[1] Google AI for Developers. Function calling with the Gemini API. https://ai.google.dev/gemini-api/docs/function-calling

[2] Google Gen AI SDK documentation. https://googleapis.github.io/python-genai/

[3] OpenAI Developers. Function calling guide. https://developers.openai.com/api/docs/guides/function-calling

[4] OpenAI Developers. Tools guide. https://developers.openai.com/api/docs/guides/tools

[5] Model Context Protocol. Server concepts. https://modelcontextprotocol.io/docs/learn/server-concepts

[6] Model Context Protocol Specification. Server tools. https://modelcontextprotocol.io/specification/2025-06-18/server/tools

[7] Google Cloud. Agent Development Kit overview. https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk

[8] Google ADK documentation. OpenAPI tools. https://adk.dev/tools-custom/openapi-tools/

[9] OpenAPI Initiative. OpenAPI Specification. https://spec.openapis.org/oas/latest.html

[10] OWASP. Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/